carterkozak and ot[bot]

Remove dead link from testing-software best practices readme (#11...

c154a34 · 7 years ago

Name	Name	Last commit date
..		
readme.md	Remove dead link from testing...	7 years ago

readme.md

Tests: Unit, Integration, ETE

A very brief introduction to software testing. This guide covers the following topics:

- Unit Tests
  - Writing Effective Unit Tests
  - Writing Testable Code
  - Resources and Further Reading
- Integration Tests
- End-to-end (ETE) Tests

Further reading: Steve Freeman, Growing Object-Oriented Software, Guided by Tests

---

Software tests validate that newly-written code functions as intended. Surprisingly, this is *not* the main reason for testing software. More importantly, **tests ensure that code continues to be correct under future modifications** and **provide documentation** for the implemented functionality.

In this guide, we distinguish three types of tests:

- **Unit tests** validate that all code paths of individual functions work as expected.
- **Integration tests** verify the interplay of a collection of modules.

- **End-to-end (ETE) tests** test entire workflows of a product.

Since the number of possible code paths typically grows exponentially with size of the tested code base, *exhaustive* tests are only possible for the smallest code units, typically functions or methods. Integration and ETE tests can never validate all cases and therefore focus on a subset of *representative cases*.

Unit Tests

The examples in this section are written in Java using the standard JUnit and AssertJ libraries for which a plethora of tutorials are available. Conceptually similar approaches exist in other languages.

Unit tests validate that all code paths of small code units work as expected:

  - Functions or methods that don't depend on a large amount of external state (e.g., the state of their class) should be tested for parameter combinations that exercise all code paths.
  - Methods whose behavior depends strongly on the state of their class should be tested for all structurally different states, again in order to test all code paths.

If you find that a particular class or method is difficult to test because you cannot freely set the parameters and state, then this usually indicates a poorly designed class or code. **Test-driven development** suggests to *first* write tests and then fill in the implementation; consequently, the implementation is necessarily designed such that it can be easily tested. Also see Writing Testable Code below.

Unit tests are typically executed before every build, both locally and on the CI servers. In order to make this feasible, consider the following guidelines:

- Branches shared by more than one developer (e.g., `develop`, `master`, `feature/...`) should never fail unit tests. Your Continuous Integration system should refuse to merge a branch with failing tests, and overriding this behavior is bad practice.
- Unit tests need to be fast and light on computational resource. In particular, do not abuse unit tests for performance tests or testing of long-lived services.

Since unit tests guard from regression bugs, be very careful and verify with the code owner **and users** before you change the behavior of a unit test. For example, if the author of a sorting algorithm implements a test that verifies the *stability* of the sorting procedure, do not remove this test when you implement a performance improvement that cannot always guarantee that sorting is stable. Users of of the sorting procedure may rely on its stability and their code will break if this assumption is violated.

Writing Effective Unit Tests

Test public interfaces. All public interfaces should be covered by unit tests. Sometimes, internal methods may require additional explicit tests; in such cases, mark the tested methods package-private and `@VisibleForTesting`.

Find reasonable names. Test names should indicate the tested code path and the expected functionality; this makes it easy to understand what exactly got broken by a particular change.

Make unit test readable. Since unit tests serve as documentation for code, they should be easy to read and understand. Testing frameworks such as JUnit provide static methods that make for very literal testing code:

```
assertThat(MyUtilityClass.upperCase("test string")).isEqualTo("TEST STRING");
```

Test each code path separately. Each test should exercise one code path or distinct, atomic behavior.

```
// BAD. Don't do this.
@Test
public void testSorting() {
    assertThat(MySort.sort(ImmutableList.<String>of())).isEmpty();
    assertThat(MySort.sort(ImmutableList.of("a"))).isEqualTo(ImmutableList.of(
    assertThat(MySort.sort(ImmutableList.of("c", "b", "a"))).isEqualTo(Immutat
    assertThat(MySort.sort(ImmutableList.of(3, 2, 1))).isEqualTo(ImmutableList

    // Test that sorting is stable.
    String a1 = "a";
    String a2 = "a";
    String a3 = "a";
    List<String> sorted = MySort.sort(ImmutableList.of(a1, a2, a3));
    assertSame(a1, sorted.get(0));
    assertSame(a2, sorted.get(1));
    assertSame(a3, sorted.get(2));
}
```

```
// Good.
@Test
public void testSorting_emptyList() {
    assertThat(MySort.sort(ImmutableList.<String>of())).isEmpty();
}

@Test
public void testSorting_singletonList() {
    assertThat(MySort.sort(ImmutableList.of("a"))).isEqualTo(ImmutableList.of(
}

@Test
public void testSorting_standardCases() {
    assertThat(MySort.sort(ImmutableList.of("c", "b", "a"))).isEqualTo(Immutat
    assertThat(MySort.sort(ImmutableList.of(3, 2, 1))).isEqualTo(ImmutableList
    assertThat(MySort.sort(ImmutableList.of(2, 1, 3))).isEqualTo(ImmutableList
}

@Test
public void testSorting_isStable() {
    String a1 = "a";
    String a2 = "a";
```

```
    String a3 = "a";
    List<String> sorted = MySort.sort(ImmutableList.of(a1, a2, a3));
    assertSame(a1, sorted.get(0));
    assertSame(a2, sorted.get(1));
    assertSame(a3, sorted.get(2));
}
```

It is OK to group several functionally similar test cases in one test:

```
@Test
public void testEmailParsing_validAddresses() {
    assertTrue(myParser.isEmailAddress("a@b.com"));
    assertTrue(myParser.isEmailAddress("real.name@company.com"));
    assertTrue(myParser.isEmailAddress("nickname@sub.domain.company.com"));
    assertTrue(myParser.isEmailAddress("name@country-domain.de"));
    // ...
}

@Test
public void testEmailParsing_invalidAddresses() {
    assertFalse(myParser.isEmailAddress(""));
    assertFalse(myParser.isEmailAddress("@"));
    assertFalse(myParser.isEmailAddress("company.com"));
    assertFalse(myParser.isEmailAddress("@company.com"));
    assertFalse(myParser.isEmailAddress("name"));
    assertFalse(myParser.isEmailAddress(null));
    // ...
}
```

Test standard, edge, and failure cases. A code unit should be tested for all error conditions, all edge cases, and a number of "standard cases." For example, a function `int[] subtractVectors(int[] left, int[] right)` that computes `left - right` for two vectors of the same cardinality should contain tests for:

- Error conditions:
  - `left` and `right` have different sizes
  - at least one of `left` or `right` is `null`
- Edge cases:
  - `left` and `right` have length 0 or 1
  - (arguably:) a subtraction overflows
- Standard cases: For example:
  - `[2, 4, 9] - [1, 2, 3] == [1, 2, 6]`
  - `[0, 0, 0] - [1, 2, 3] == [-1, -2, -3]`

Use matchers instead of implementing equals(). Override `equals()` when mandated by application or library logic, but *never* implement `equals()` merely in order to simplify your life in unit tests. Use AssertJ matchers instead. This may seem cumbersome at first, but the net benefit is huge as soon as more than one property of an object is tested or there is more than one test.

Use mocks. Use mock objects to construct the context and state required to test a particular code path. Mock objects are cheap, fast, explicit, and concise, and it is almost always more stable and easier to set up a mock object than to instantiate and populate a full implementation of a particular class or interface. Testing functionality with real implementations is the duty of integration or end-to-end tests. See the section on Writing testable code for an example.

Writing Testable Code

Further reading: Miško Hevery, Guide: Writing Testable Code

Making methods, classes, or modules easy to test is a lot of work and may seem like a cumbersome exercise to a novice. Fortunately and coincidentally, code that is easy to test is usually considered *better* along several, not necessarily testing-related, dimensions: it is more reusable, more modular, easier to understand, easier to refactor, and less tightly coupled.

Probably the most important design consideration is to **inject dependencies** into a class through the constructor instead of letting a class construct its dependencies itself. Let's look at a simple, but drastic, example:

```
public final class Backend {
    private final Database database;

    public Backend(String dbHost, String dbUsername, String dbPassword) {
        this.database = new OracleDatabase(dbHost + ":9271", dbUsername, dbPas
    }

    private boolean isValidUser(User user) {
        return user.getAccountExpiryDate().isBeforeNow() && user.isActive();
    }

    public List<String> getValidUsers() {
        List<String> users = Lists.newArrayList();
        for (User u : database.getAllUsers()) {
            if (isValidUser(u)) {
                users.add(u.getUsername());
            }
        }

        return users;
    }
}
```

This class uses a `Database` instance to retrieve `User` objects in the `getValidUsers()` method; the retrieved users are then filtered down to the list of active users and returned by their user names. The database connection is configurable through the `dbHost`, `dbUsername`, and `dbPassword` parameters that are used to instantiate a corresponding `OracleDatabase` object. A unit test could look like this:

```
@Test
public void testGetValidUsers() {
    Backend backend = new Backend("localhost", "admin", "password");
    assertThat(backend.getValidUsers()).isEqualTo(ImmutableList.of("vanessa",
}
```

Unfortunately, **this is a terrible test** for several reasons:

- Even though the database can be configured through the host/username/password parameters, the test can only succeed when a valid Oracle database instance is available. Even if this may be the case on a development machine, it is probably not on the CI servers.
- It is entirely unclear what the test actually asserts since the return value of `getValidUsers()` depends entirely on (a) the current time (via `isBeforeNow()` in `isValidUser()`), and (b) the contents of the Oracle database. Unit tests need to be self-contained and the tested behavior must be self-explanatory.

We can improve the test by first rewriting the code:

```
public final class Backend {
    private final Database database;
    private final Predicate<User> validUserFilter;

    @VisibleForTesting
    Backend(Database database, Predicate<User> validUserFilter) {
        this.database = database;
        this.validUserFilter = validUserFilter;
    }

    public List<String> getValidUsers() {
        List<String> users = Lists.newArrayList();
        for (User u : database.getAllUsers()) {
            if (validUserFilter.apply(u)) {
                users.add(u.getUsername());
            }
        }

        return users;
    }
}

/**
 * Factory methods for creating {@link Backend} instances.
 */
public class Backends {
    public static Backend withOracleCredentials(String dbHost, String dbUserna
        Database oracleDatabase = new OracleDatabase(dbHost + ":9271", dbUserr
        return Backend.withDatabase(oracleDatabase);
    }

    public static Backend withDatabase(Database database) {
        return new Backend(database, new Predicate<User>() {
            @Override
            boolean apply(User user) {
```

```
                return user.getAccountExpiryDate().isBeforeNow() && user.isAct
            }
        );
    }
}
```

We have introduced two important changes:

- The `Database` is injected through the constructor instead of being created in the constructor.
- The `isValidUser()` method is now a `Predicate<User> validUserFilter`.

Note that the (single) constructor is the most generic way of creating new Backend instances, and that the constructor does zero work. It may however be cumbersome and error-prone to use as it requires an explicitly given `validUserFilter`. In order to make the class easier to use, we provide two static factory methods, one for creating a Backend with an Oracle database (`withOracleCredentials(...)`), and one for a generic Database (`withDatabase(...)`). If we wanted to enforce the use of the canonical `validUserFilter`, we could mark the constructor package-private and annotate it with `@VisibleForTesting`; the latter serves merely as documentation. (C++ has the notion of *friend classes* [i.e., "private with exceptions"] that can be used to statically enforce that only test classes may use this constructor.)

The refactored class can be tested much more easily because its state can be constructed explicitly in order to verify a specific code path or part of the method or class behavior. For example:

```
@Mock
public Database mockDatabase;

@Test
public void testGetValidUsers_WhenNoUserInDatabase() {
    when(mockDatabase.getAllUsers()).thenReturn(ImmutableList.<User>of());
    Backend backend = new Backend(mockDatabase, Predicates.alwaysTrue());
    assertThat(backend.getValidUsers(), is(emptyCollectionOf(String.class)));
}

@Test
public void testGetValidUsers_ExpiredUsersAreNotReturned() {
    when(mockDatabase.getAllUsers()).thenReturn(ImmutableList.of(
        new User("vanessa", DateTime.now().plusDays(1)),
        new User("peter", DateTime.now().plusDays(2)),
        new User("jack", DateTime.now().minusDays(1)) // account expires 1 day
    ));
    Backend backend = Backend.withDatabase(mockDatabase);
    assertThat(backend.getValidUsers(), is(ImmutableList.of("vanessa", "peter"
}

...
```

Note that through dependency injection, we are able to programmatically construct a "database" that contains exactly the records we need in order to test certain code paths. For example, we can dynamically assign account expiry dates. As promised earlier, the class is now more flexible and less tightly coupled to a particular database implementation (e.g., `OracleDatabase`).

For more on this topic, please see Miško Hevery's *Guide: Writing Testable Code*.

Resources

- Miško Hevery, Guide: Writing Testable Code
- Miško Hevery, The Clean Code Talks -- Unit Testing
- Lisa Crispin, Agile Testing: A Practical Guide for Testers and Agile Teams

Integration Tests

(Scheduled to be available in Baseline release v2.0.0.)

End-to-end (ETE) Tests

(Scheduled to be available in Baseline release v2.0.0.)